EXPERIMENT NO. 2

Version Control Operations Using Git

Practical Record • Step-by-step instructions

1. Aim

To study and perform basic version control operations using Git, including repository initialization, staging, committing, branching, merging, viewing differences, and connecting a local repository to GitHub.

2. Requirements

• Git for Windows • Command Prompt (CMD) or Git Bash • GitHub account (optional for local operations)

3. Step-by-Step Procedure

Step 1: Install Git

1. Open the official Git installation page: https://git-scm.com/install/.
2. Download and install Git for Windows using the recommended options.
3. Open Command Prompt and verify the installation:

```
git --version
```

Expected output: a version string such as `git version 2.x.x`. The exact version may differ.

Step 2: Configure Git

Replace the sample name and email with your preferred commit identity:

```
git config --global user.name "Your Name"
git config --global user.email "your_email@example.com"
git config --list
```

Purpose: Git records the configured name and email as the author information for commits.

Step 3: Create a project folder and file

```
mkdir GitExperiment
cd GitExperiment
echo Hello Git > hello.txt
type hello.txt
```

Expected output: `Hello Git`.

Step 4: Initialize a repository

```
git init
git status
```

Purpose: `git init` creates a local repository. The new file should initially appear as untracked.

Step 5: Stage a file

```
git add hello.txt
git status
```

Expected result: `hello.txt` appears under “Changes to be committed.”

Step 6: Commit the changes

```
git commit -m "Initial commit"
git log --oneline
```

Purpose: A commit saves a snapshot of the staged changes in the repository history.

Step 7: Modify the file and commit again

```
echo This is my first Git experiment >> hello.txt
type hello.txt
git status
git add hello.txt
git commit -m "Update hello.txt"
git log --oneline
```

Expected result: the history contains two commits.

Step 8: Create and switch to a branch

```
git branch feature
git switch feature
git branch
```

Purpose: A branch lets you develop a feature separately from the main line of work. The current branch is marked with an asterisk (*).

Step 9: Commit a feature and merge it

```
echo New feature added > feature.txt
git add feature.txt
git commit -m "Add new feature"
git switch main
git merge feature
```

If the repository's default branch is named `master`, use `git switch master` instead of `git switch main`. After merging, check the files with `dir`.

Step 10: View file differences

```
echo Another line added >> hello.txt
git diff
git add hello.txt
git diff --staged
```

Purpose: `git diff` shows unstaged changes, while `git diff --staged` shows staged changes.

Step 11: Connect to GitHub (optional)

1. Create a new repository on GitHub named `GitExperiment`. For this workflow, do not initialize it with a README or other files.
2. Copy the repository URL and run the following from your local project folder:

```
git remote add origin YOUR_REPOSITORY_URL
git branch -M main
git push -u origin main
```

Replace `YOUR_REPOSITORY_URL` with the actual repository URL. Complete any authentication prompts. This uploads the current main branch to GitHub.

4. Important Git Commands

Command	Purpose
`git init`	Initialize a repository
`git status`	Check repository status
`git add .`	Stage changes in the current directory
`git commit -m "message"`	Commit staged changes
`git log`	View commit history
`git branch`	List branches
`git switch branch_name`	Switch branches
`git merge branch_name`	Merge a branch
`git diff`	Show unstaged differences
`git clone URL`	Copy a remote repository locally
`git pull`	Fetch and integrate remote changes
`git push`	Upload local commits to a remote

5. Result

Basic version control operations using Git were performed: repository initialization, staging, committing, modifying files, branching, merging, and viewing differences. The local repository can also be connected to GitHub for remote storage.

6. Conclusion

Git helps track changes in files, maintain project history, and support collaborative software development through branches, commits, and remote repositories.